

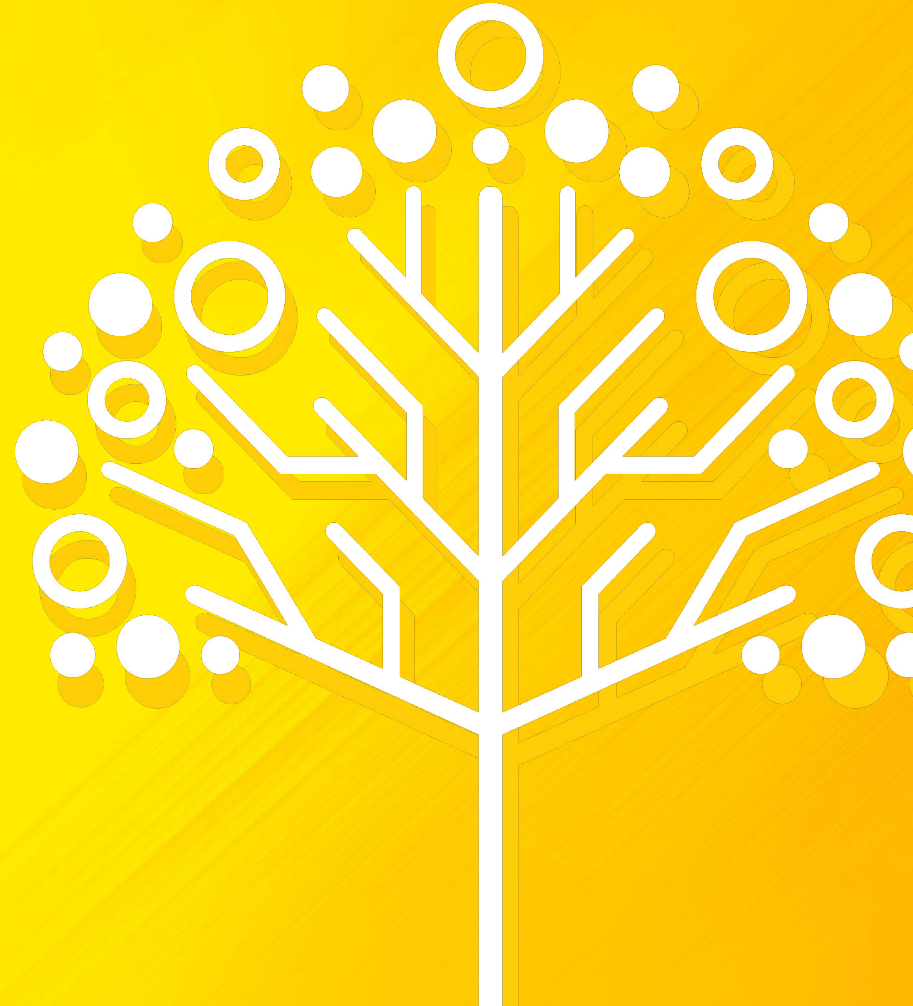


Michigan
Technological
University

UAV Collision Detection & Prevention

Final Presentation

Group Members: Felicia Huffman,
Sam Weiss, Jacob Brendemuehl,
and Olivia Gette



Background

- LiDAR applications were first developed in the early 1960's shortly after the invention of the laser beam
 - Early applications were for tracking airplanes
- Since then, use cases have expanded to agriculture, archaeology, atmosphere, military, law enforcement, and many other areas
- Recently, it has become a big part of the AI world with its use in autonomous vehicles and now, it's use in unmanned aerial vehicles (UAVs)
- UAV's have been around for a little over a century now, but only recently have been developed to support non-military use, such as package delivery

Project Overview

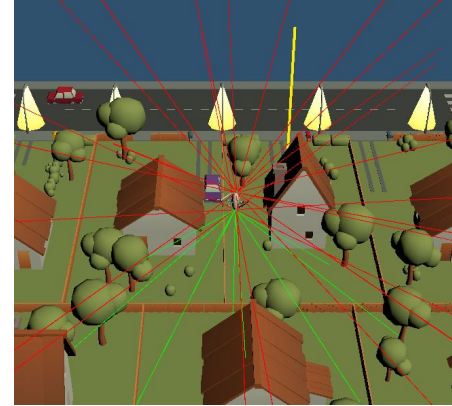
- With more use cases for LiDAR based sensing applications emerging daily, it is important to fill the research gap surrounding LiDAR's practicality in unknown environments
- Soon, many simple tasks, such as delivering a package, will be automated through drone delivery
- Our goal is to provide a baseline for other researchers and prove that it is possible for an agent to navigate an unknown environment using solely LiDAR based detection systems

Unity Scene & LiDAR Setup

- Unity is a cross-platform real-time 3D development platform
 - We are using Unity to simulate the environment for drone collision
- Unity allows us to create/manage assets, scripts, and environments to manipulate our drone and the items around it
- Our Unity Environment currently is a neighborhood mockup from a unity asset pack on the Unity store
- This mockup includes objects such as trees, houses, cars, people, roads, etc. to help create a realistic environment for the drone to train in and complete tasks.
- We have also implemented moving birds, and a wind feature as well.

Unity Scene & LiDAR Setup cont.

- LiDAR is a remote sensing method that uses pulses of laser light to measure distances and precise 3D models of objects and environments.
- Our drone is utilizing LiDAR to detect and report objects and surfaces that are close to or in the path of the drone
- Our drone has a sensor on the top and bottom that emit 2 hemispheres of lasers
 - Each sensor emits 32 beams, full cluster is 64
- The lasers detect and record the surrounding environment in real time and reports the data to be processed.
- In the Unity environment, you can see a visual representation of the beams where green beams = hit, red = no hit.



Agent Logic & Rewards Part I

```
! drone_goal.yaml X DroneAgent.cs DroneAutopilot.cs LiDARLogger.cs
config > ! drone_goal.yaml
1 # config/drone_goal.yaml
2 # file path is load bearing ^
3
4 default_settings:
5   trainer_type: ppo
6   hyperparameters:
7     batch_size: 2048
8     buffer_size: 40960
9     learning_rate: 3.0e-4
10    beta: 3.0e-3
11    epsilon: 0.2
12    lambda: 0.95
13    num_epoch: 3
14    learning_rate_schedule: linear
15  network_settings:
16    normalize: true
17    hidden_units: 256
18    num_layers: 3
19  reward_signals:
20    extrinsic:
21      gamma: 0.995
22      strength: 1.0
23    time_horizon: 256
24    max_steps: 1e7
25    summary_freq: 10000
26
27 behaviors:
28   drone_goal: {}
29   # leaving this {} empty causes Unity to use default_settings ^
30
```

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M **do**

Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ **do**

With probability ε select a random action a_t

otherwise select $a_t = \arg\max_a Q(\phi(s_t), a; \theta)$

Execute action a_t in emulator and observe reward r_t and image x_{t+1}

Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

Every C steps reset $\hat{Q} = Q$

End For

End For

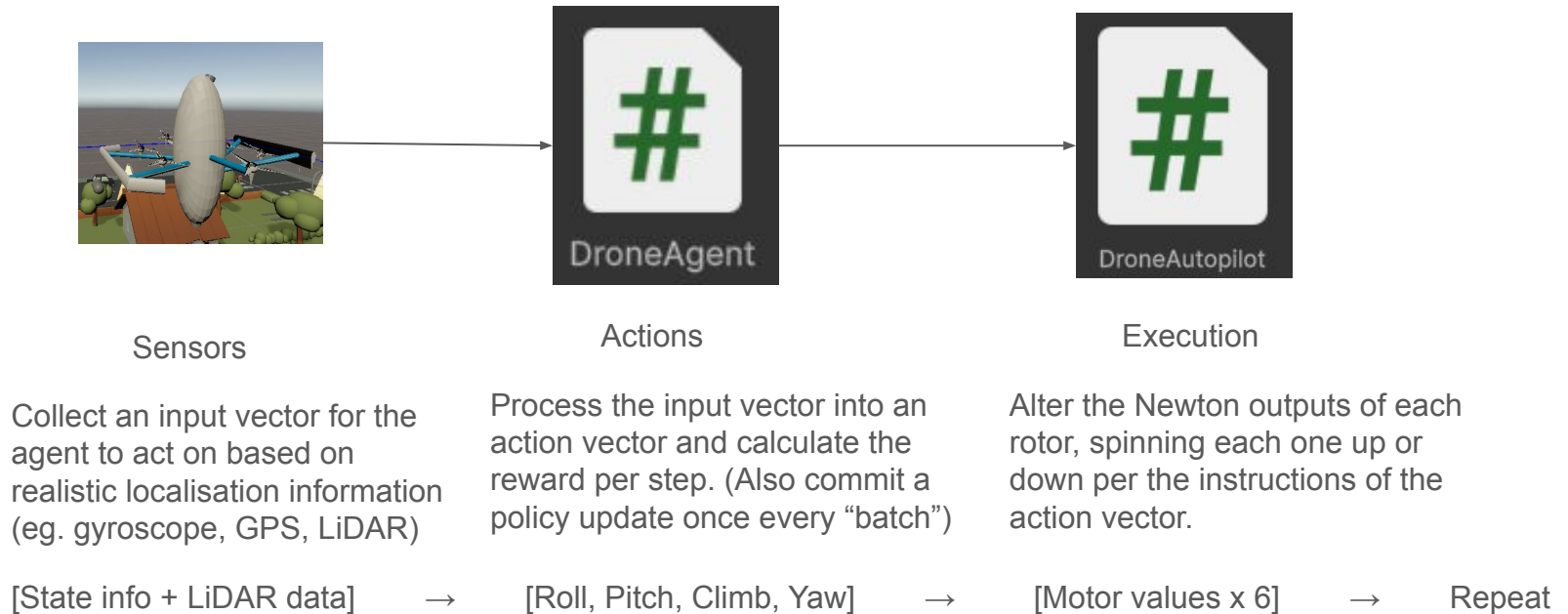
Agent Logic & Rewards Part II

1. Each step, if $\text{currDistance} < \text{prevDistance}$, get a small reward.
 2. Each step, take currVelocity , draw a vector toward the goal, small reward if close
 3. Each step, incur a small “time” penalty.
 4. Each step, if dronePos is near goalPos , get a small reward.
 5. Each step, take $\min(\text{LiDAR_distance})$, get an inversely negative reward to it.
-

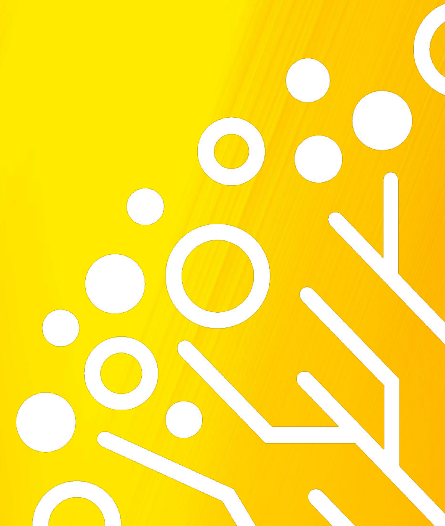
1. If the drone is “stuck” too long, end the episode and get a static negative reward
2. If the drone crashes, end the episode and get a negative reward as scaled by crash speed
3. If the drone reaches its goal, end the episode and get a large static reward

Training Pipeline

- Sourcing observations required as inputs for Deep Q-Learning policy updates from was done by a simple three step feedforward algorithm:

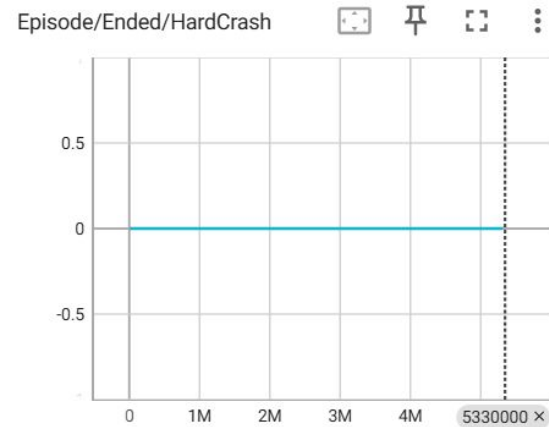


Results and Analysis

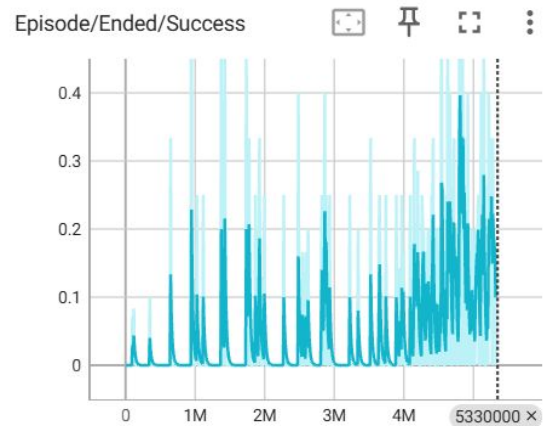


Reward Curves

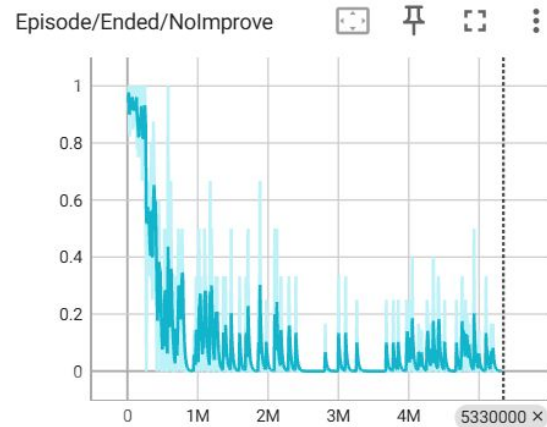
- Reward curve represents how the drone's performance improves throughout training
- Early episodes show low or negative rewards due to crashes and unstable flight
- As training progresses, rewards trend upward as it learns more controlled motion
- Rising reward curve means the agent is learning consistently



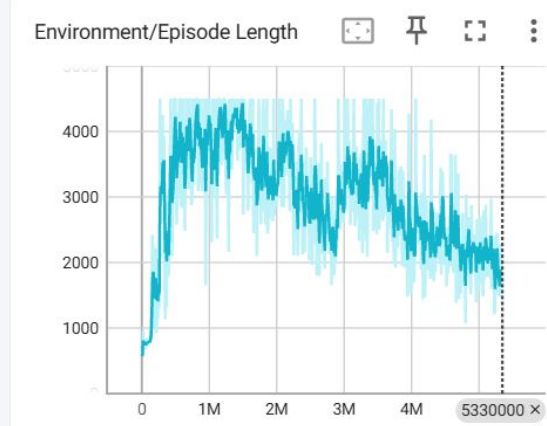
Run ↑	Smoothed	Value	St
drone_goal_401\drone_goal	0	0	5;



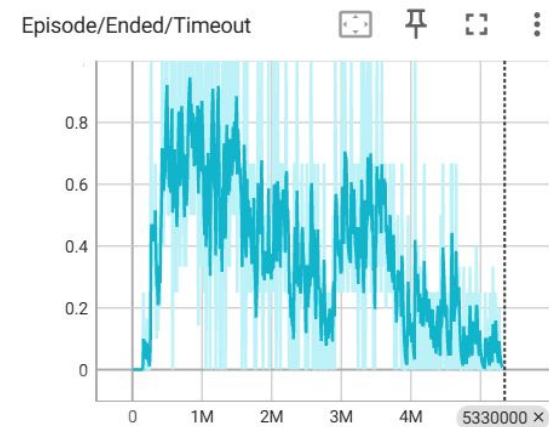
Run ↑	Smoothed	Value	St
drone_goal_401\drone_goal	0.0995	0	5;



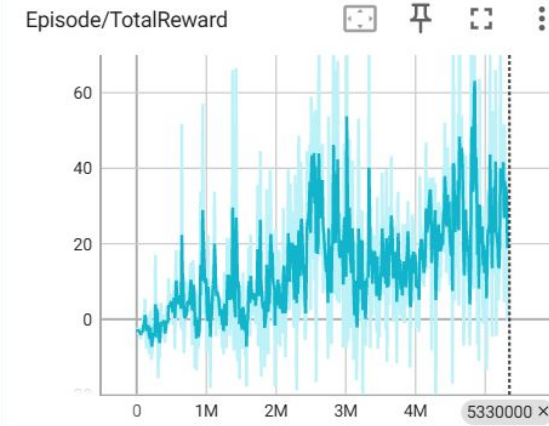
Run ↑	Smoothed	Value	St
drone_goal_401\drone_goal	0.0002	0	5;



Run ↑	Smoothed	Value	St
drone_goal_401\drone_goal	1,894.1788	2,284	5;



Run ↑	Smoothed	Value	St
drone_goal_401\drone_goal	0.0072	0	5;



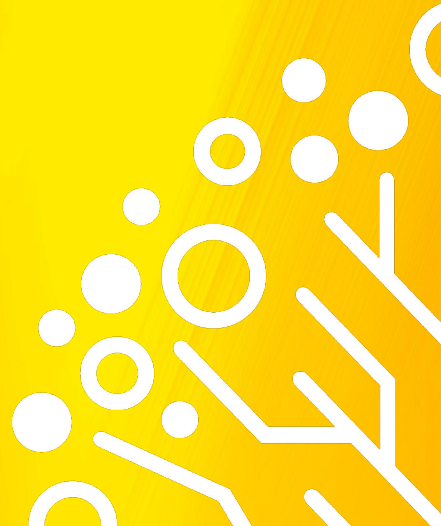
Run ↑	Smoothed	Value	St
drone_goal_401\drone_goal	18.8068	-0.5021	5;

Michi

Interpretation

- Success rate increases over time as the drone becomes better at reaching the goal
- No hard crashes ended episodes, but lower number of episodes ended due to timeout or no improvements
- These metrics help verify whether improvements in reward correspond to meaningful behavioral changes
- Shorter episode lengths could be due to more/quicker successes
- These results suggest that the agent is learning navigation and stabilization skills

Challenges and Lessons Learned



Challenges with LiDAR & Unity Environment

- Challenges with Unity included:
 - Learning how to use, familiarizing with the software
 - Merge conflicts with multiple collaborators working on the project
 - Pushing and Pulling simultaneously breaking the project
 - Breaks in scripts causing the whole project to malfunction
- Challenges with LiDAR included:
 - Learning how LiDAR works
 - Trying to implement real LiDAR vs unity cheating
 - Self-Collision, beams hitting the drone itself
 - Getting accurate data from LiDAR to train the drone

Challenges with Reinforcement Learning

- Many of the challenges we faced were familiar to any RL task:
 - High compute requirements and waiting periods
 - Poor real time feedback for changes
 - Reward hacking/unexpected behavior
 - Opaque/black-box nature of policy
 - Difficulty collaborating



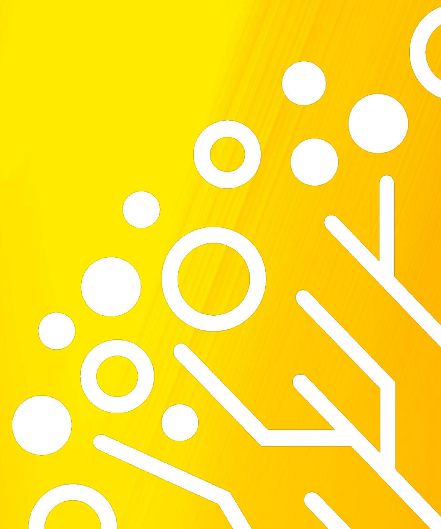
Challenges with Project as a Whole

- Getting success and collision detection to work reliably in Unity and ML-Agents
- Designing an effective reward function without causing unintended behaviors
- Ensuring stable training, many runs showed noisy reward curves or slow convergence
- Managing environment setup issues, including physics accuracy and consistent episode resets
- Version control, software compatibility and other classic software development woes
- Unity doesn't vibe with MTU's HPC (specifically, it's headless mode is poorly supported by the library we used ML-Agents)

Future Modifications

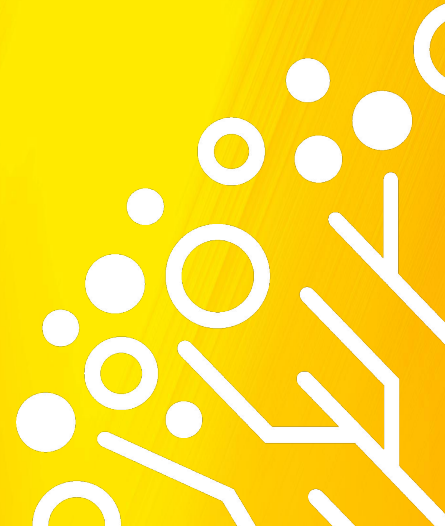
- Find a way to get access to an HPC to run a more complex model
- Extend the algorithm to a real-world application by embodying it with a drone
- Create new and more challenging environments to navigate, such as a maze
- Adding a second drone to the environment to understand how the model would react
- Adding random noise and other complications to the sensors to better simulate a real world environment
- Train the model to not only deliver a package, but also return to the hub

Demo:



References:

- [1] I. Nurgaliev and Y. Eskander, "The use of drones and autonomous vehicles in logistics and delivery," **Logistics and Transport**, vol. 1–2(57–58), pp. 77–87, 2023. DOI: 10.26411/83-1734-2015-2-55-6-23.
- [2] H. Studiawan and K.-K. R. Choo, "Introduction to drone and UAV technology," in **Drone and UAV Forensics**, Cham, Switzerland: Springer, 2025, pp. 1–19. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-93511-4_1
- [3] N. Mehendale and S. Neoge, "Review on Lidar Technology," SSRN, May 18, 2020. [Online]. Available: <https://ssrn.com/abstract=3604309>
- [4] P. Seoane, E. Aldao, F. Veiga-López, and H. González-Jorge, "Assessment of LiDAR-based sensing technologies in bird–drone collision scenarios," **Drones**, vol. 9, no. 1, p. 13, Jan. 2025. [Online]. Available: <https://doi.org/10.3390/drones9010013>
- [5] E. Aldao, L. M. González-de Santos, and H. González-Jorge, "LiDAR based detect and avoid system for UAV navigation in UAM corridors," **Drones**, vol. 6, no. 8, p. 185, Aug. 2022. [Online]. Available: <https://doi.org/10.3390/drones6080185>
- [6] Q. Liang, Z. Wang, Y. Yin, W. Xiong, J. Zhang, and Z. Yang, "Autonomous aerial obstacle avoidance using LiDAR sensor fusion," **PLOS ONE**, vol. 18, no. 6, pp. 1–22, Jun. 2023. [Online]. Available: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0287177>
- [7] Atlassian, "Jira Software." [Online]. Available: <https://www.atlassian.com/software/jira>
- [8] GitHub, Inc., "GitHub: Where the world builds software." [Online]. Available: <https://github.com/>
- [9] Unity Technologies, "Unity Real-Time Development Platform." [Online]. Available: <https://unity.com/>
- [10] Blender Foundation, "Blender: Free and open source 3D creation suite." [Online]. Available: <https://www.blender.org/>
- [11] Google, "Google Drive," Google, Mountain View, CA, USA. [Online]. Available: <https://www.google.com/drive/>
- [12] Anaconda Software Distribution. Anaconda Inc. [Online]. Available: <https://www.anaconda.com/>
- [13] Project Jupyter, "Jupyter Notebook." [Online]. Available: <https://jupyter.org/>
- [14] I. Catalano, X. Yu, and J. P. Queralta, "Towards robust UAV tracking in GNSS-denied environments: a multi-LiDAR multi-UAV dataset," in **Proc. 2023 IEEE Int. Conf. Robot. Biomimetics (ROBIO)**, 2023, pp. 1–7.



Thank you!
Questions?